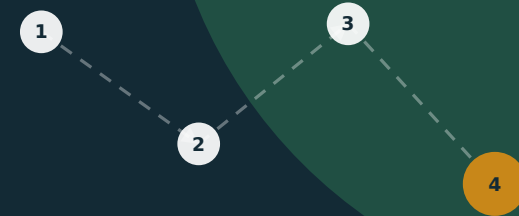


# Shadowseed Pro

Authority flow, local deployment, persistence, and trust boundaries

A readable map of how possible omissions can be remembered without silently becoming trusted guidance.



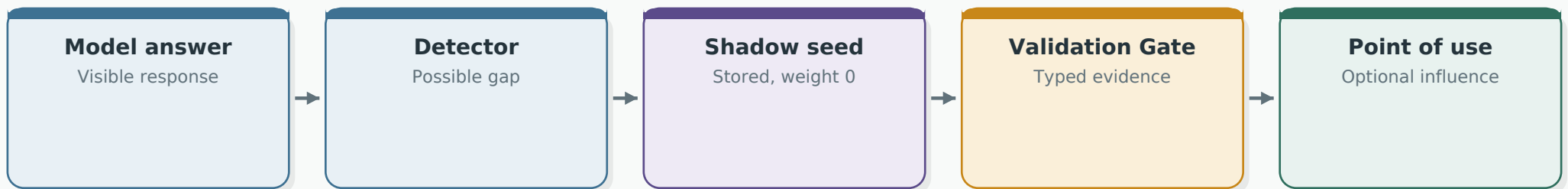
## STATUS

**Research-ready. Production-local assurance candidate.**

Not a hosted or multi-user production service. No general answer-quality claim.

# Remember without trusting

Shadowseed adds a controlled memory path beside normal model generation. A detector may record a possible omission, dependency, contradiction, or question worth investigating. The record is a candidate, not a fact.



**trace > 0**

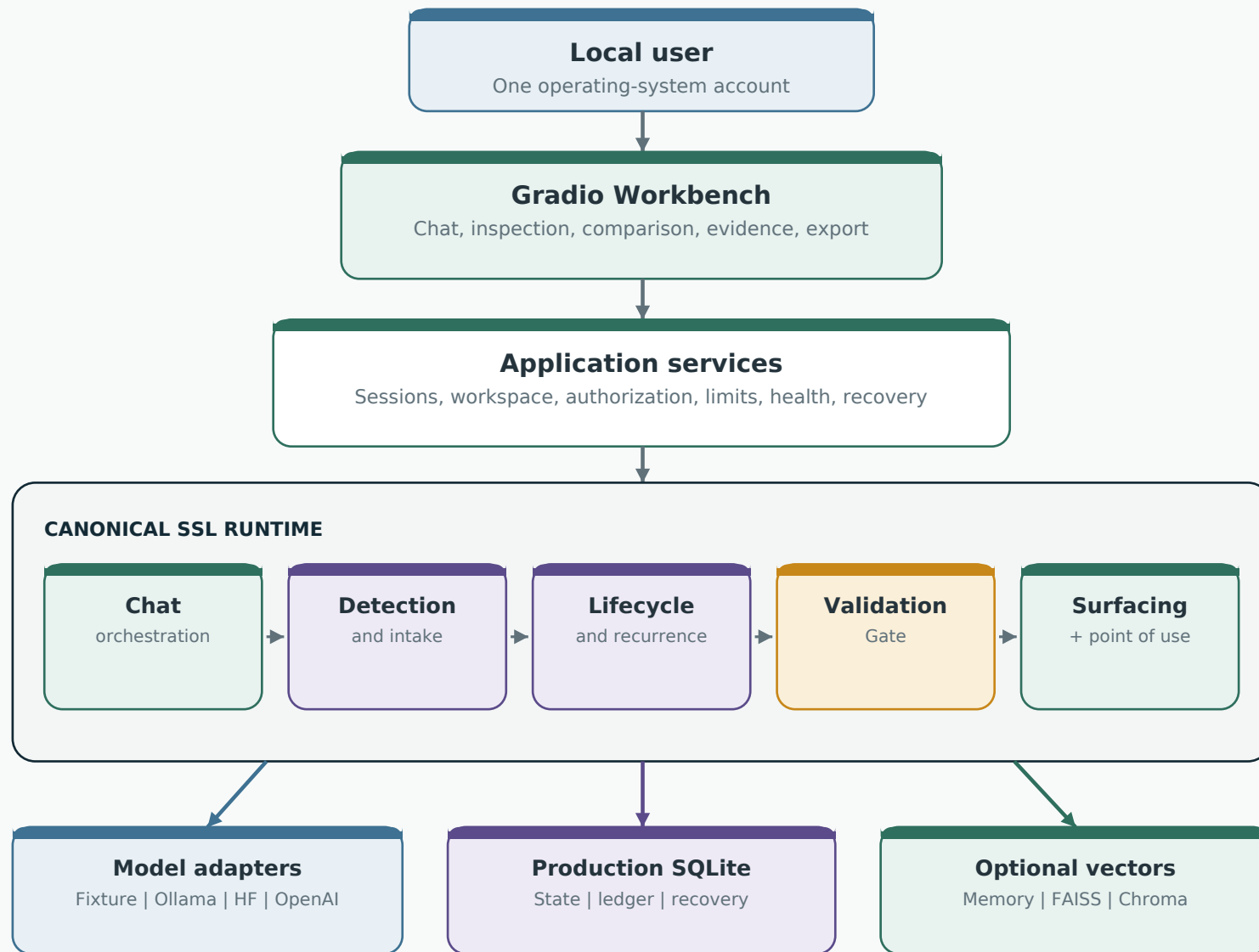
The candidate is present in shadow memory.

**weight = 0**

A new candidate has no steering authority.

**Memory and authority remain separate throughout the lifecycle.**

# The shipped product is one local application



## One authority path

The UI coordinates operations. It cannot decide that a candidate is true, write authority fields directly, or bypass the Gate.

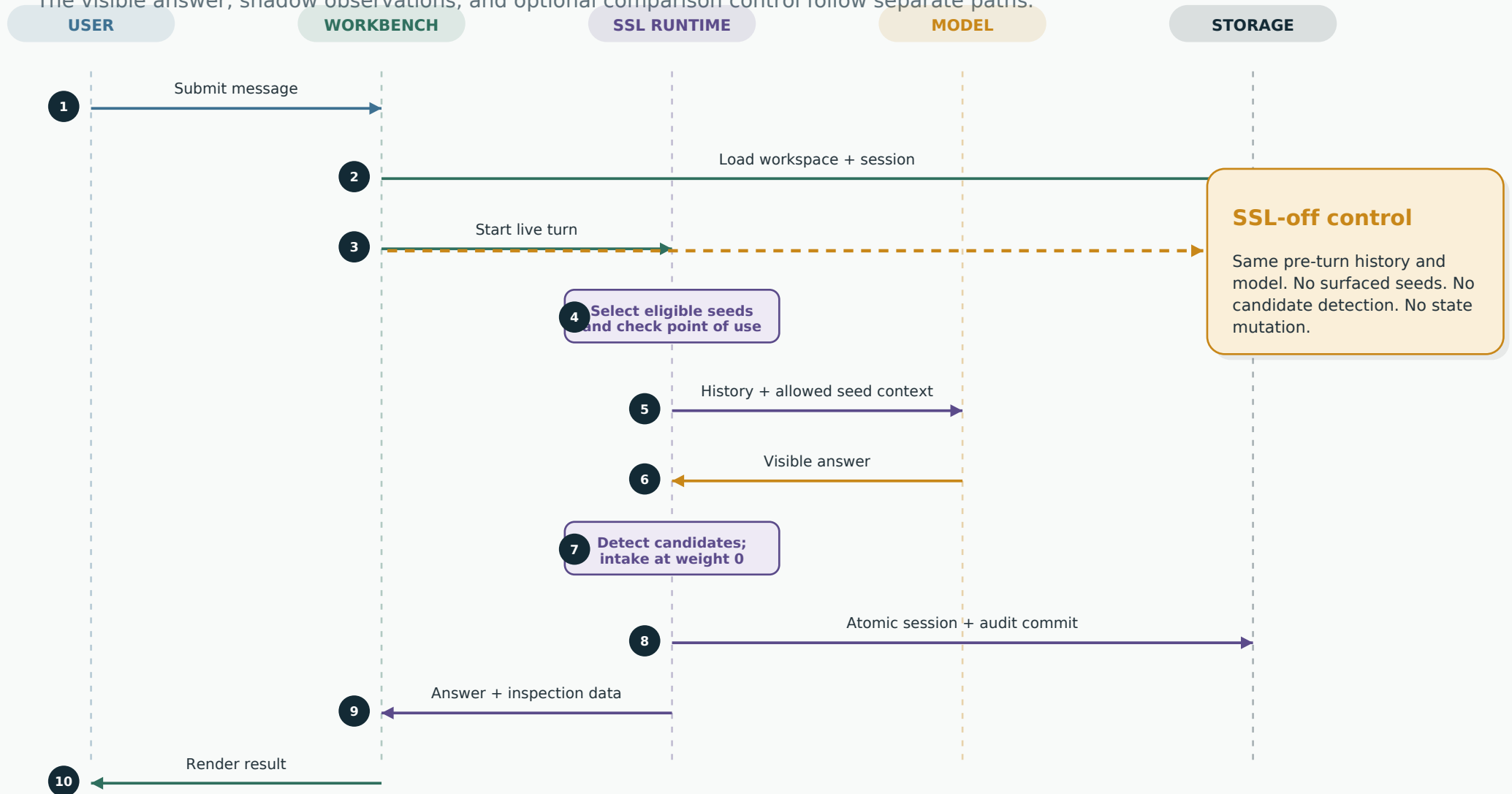
The same canonical runtime serves product chat, diagnostics, and research harnesses.

## What this is not

- Not six microservices behind an API gateway.
- Not a tenant or organization platform.
- Not a conventional document RAG service.
- Not a public-network deployment.

# One live chat turn

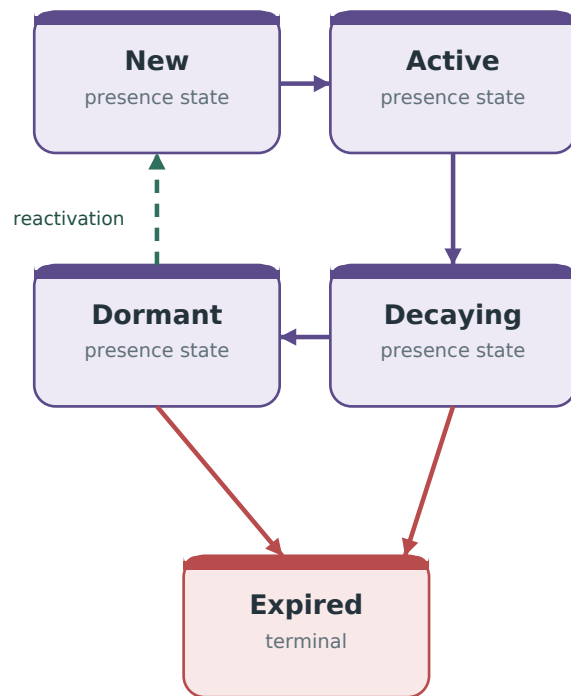
The visible answer, shadow observations, and optional comparison control follow separate paths.



# A candidate can be remembered and still remain powerless

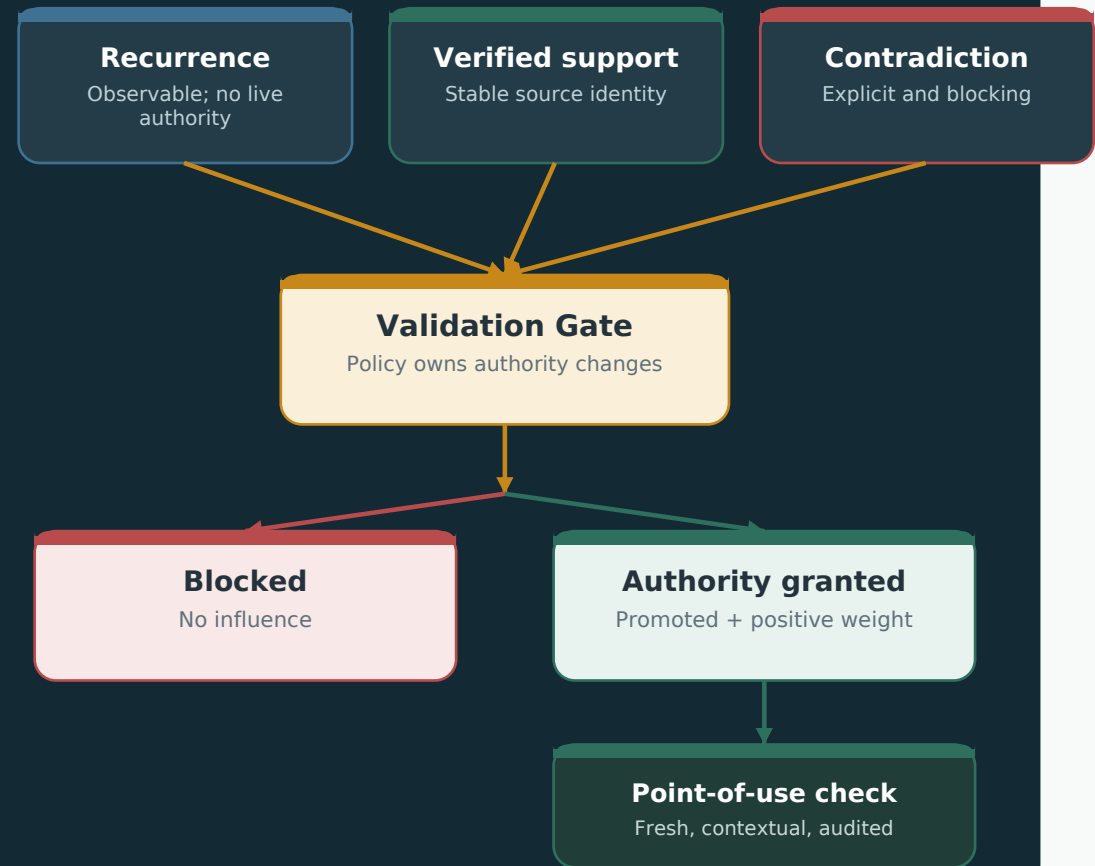
Lifecycle state describes presence. Authority state describes permission to steer. They are related, but not interchangeable.

## OBSERVATION LIFECYCLE



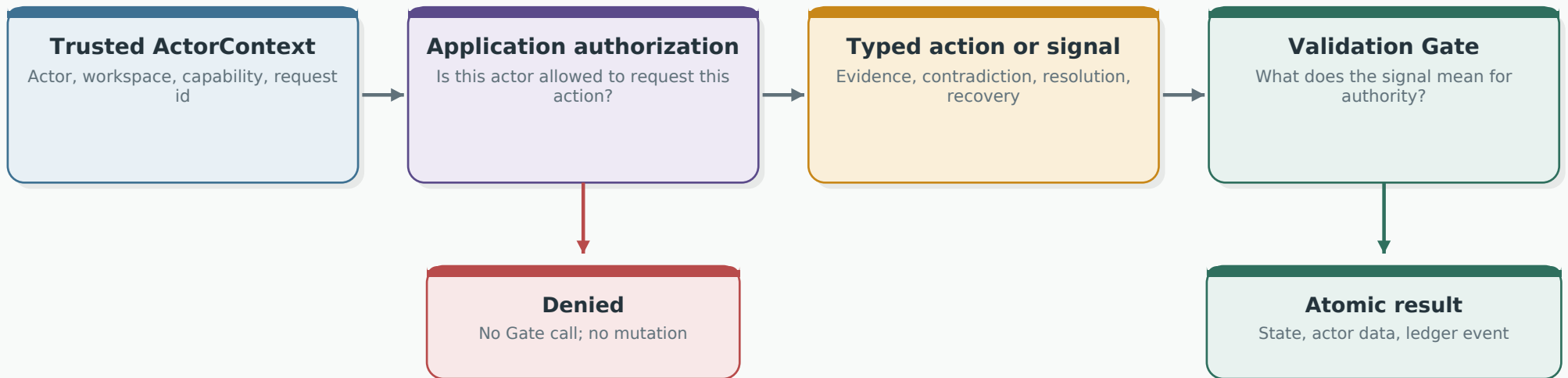
TTL reduces trace. Dormancy can reverse; expiry is terminal.

## AUTHORITY DECISION



# Permission to request is not authority to decide

The local product attributes sensitive operations to a trusted ActorContext scoped to one workspace.

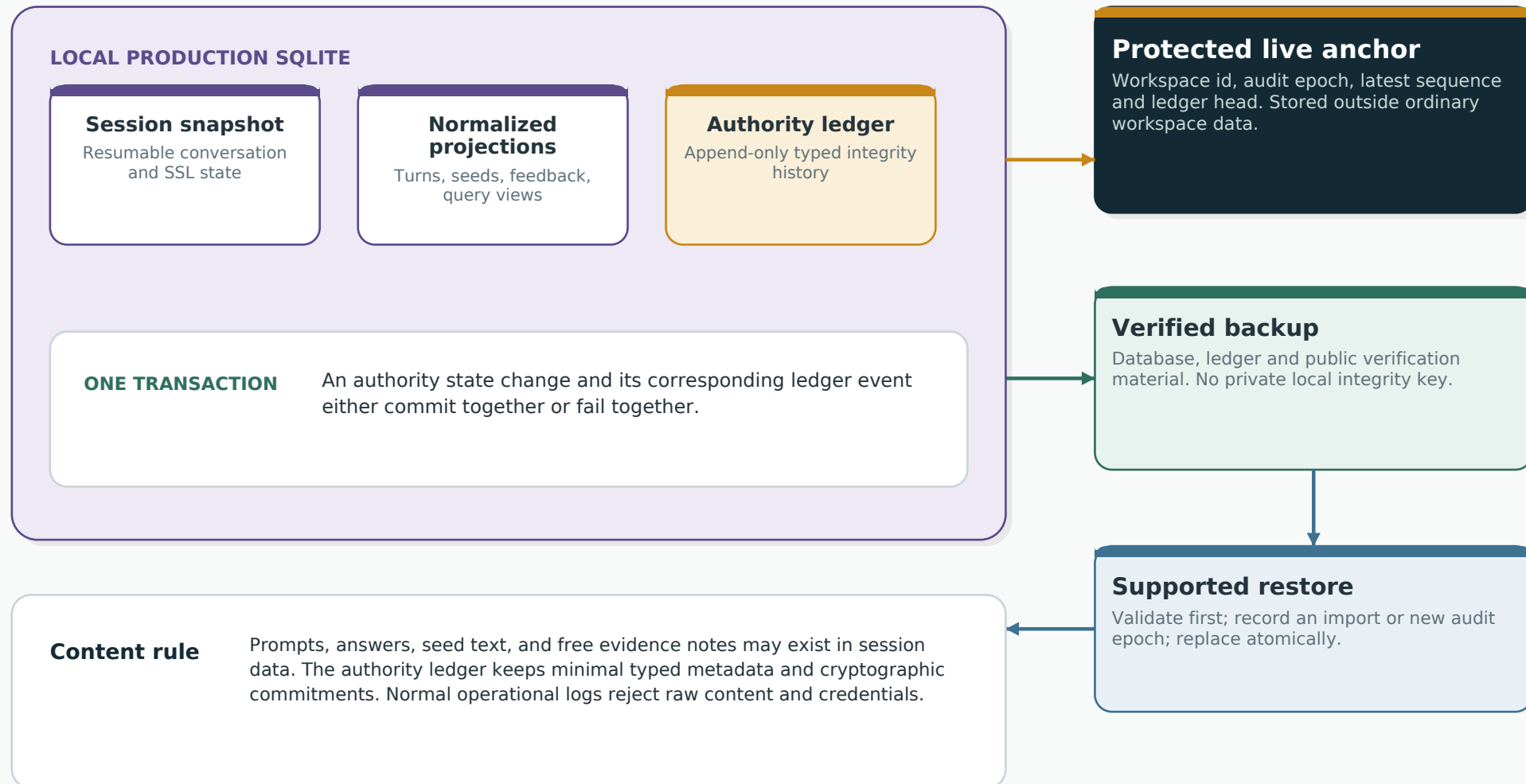


| Operation             | Capability               | Authorization permits                | It cannot do               |
|-----------------------|--------------------------|--------------------------------------|----------------------------|
| Normal chat           | chat.use                 | Start a live product turn            | Grant seed authority       |
| Verify evidence       | evidence.verify          | Submit attributable verified support | Force promotion            |
| Submit contradiction  | contradiction.submit     | Create a blocking challenge          | Edit weight directly       |
| Resolve contradiction | contradiction.resolve    | Close it with a recorded basis       | Restore authority silently |
| Backup or restore     | workspace.backup_restore | Use the supported recovery path      | Bypass integrity checks    |

**A checkbox or client-supplied boolean is never production authorization.**

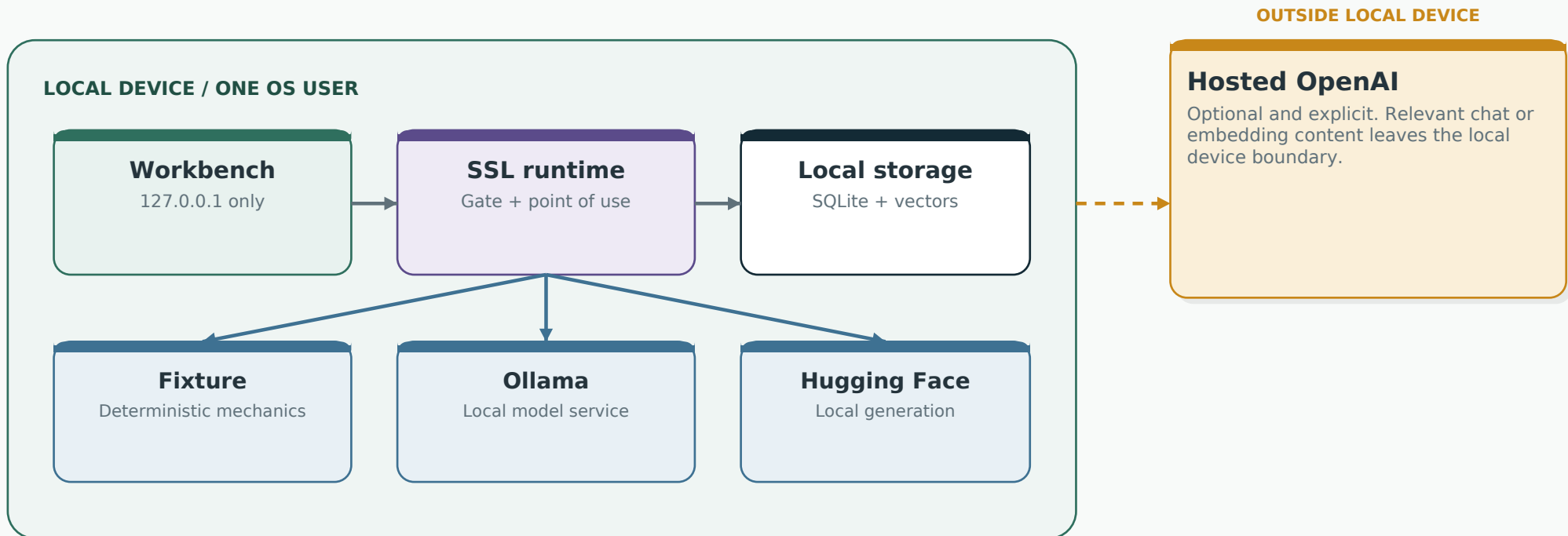
# The ledger makes authority history verifiable

Authority mutation and its ledger event commit in the same SQLite transaction. A protected anchor outside the workspace detects rollback.



# The supported product boundary is local and explicit

The production-local launcher binds to 127.0.0.1. Provider choice may move data outside that boundary, so hosted use remains explicit.

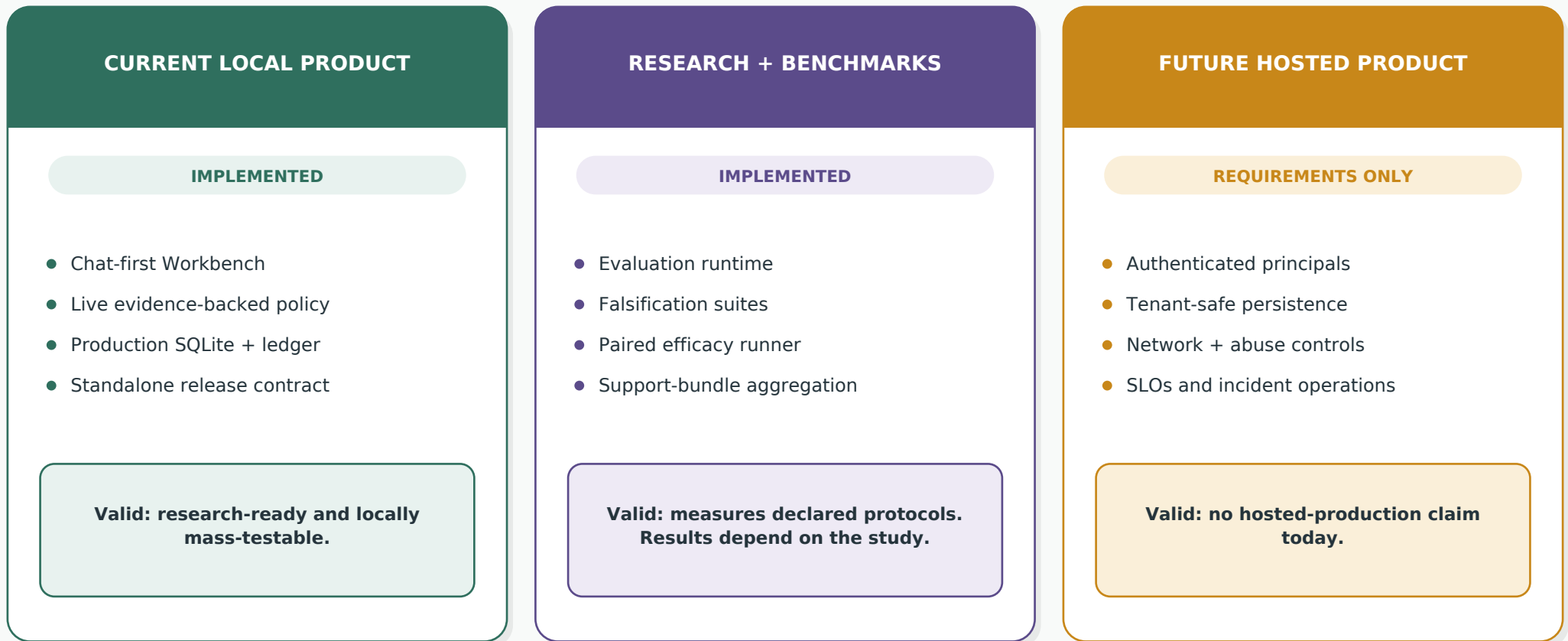


## Not included in the current claim

- No built-in multi-user login or tenant isolation
- No hostile-network TLS, CSRF, or public session boundary
- No hosted rate, cost, abuse, or administrator controls

# Three repository concerns, three different claims

Product code, research tooling, and future hosted requirements share concepts without becoming one deployment or one evidence claim.



**A functioning benchmark runner is not proof of benefit. A local UI is not a hosted security boundary.**

# How to read and maintain this design

## Canonical sources

- **Architecture overview** Runtime layers and authority model  
<docs/architecture/overview.md>
- **Lifecycle and Gate** State, signals, contradiction, point of use  
<docs/architecture/lifecycle-and-gate.md>
- **Actor authorization** Capabilities and trusted ActorContext  
<docs/architecture/production-actor-authorization.md>
- **Persistence and audit** Ledger, anchor, backup, restore, deletion  
<docs/architecture/production-persistence-and-audit.md>
- **ADR-005 / 006 / 007** Product, local boundary, future hosted boundary  
<docs/architecture/adr/>
- **Research status** Current evidence and bounded claims  
<docs/research/status.md>

## Interpretation rule

Use this document as a readable map. Use the linked specifications and contract tests for exact behavior.

**A later code or architecture change can make this snapshot stale.**

## Refresh checklist

- Check the current repository version and exact commit.
- Reconcile every diagram with canonical architecture documents.
- Verify product status and evidence claims separately.
- Regenerate the PDF and inspect every rendered page.
- Keep the Markdown source as the editable reference.

End of high-level design